

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: # -----
# 1. Load daily precipitation data
# -----

precip = pd.read_csv(
    "../Capstone/raw data/fact_Atlantic_Climate.csv",
    parse_dates=["Date/Time"],
    low_memory=False
)
```

```
In [3]: # Normalize column names
precip.columns = precip.columns.str.strip().str.replace(" ", "_")

# Adjust according to the actual column name.
precip = precip.rename(columns={
    "Climate_ID": "ClimateID",
    "Total_Precip_(mm)": "DailyPrecip_mm"
})

# Ensure consistent type
precip["ClimateID"] = precip["ClimateID"].astype(str)

# Remove days without precipitation (important for P95)
precip = precip.dropna(subset=["DailyPrecip_mm"])
```

```
In [4]: # -----
# 2. Calculate P95 per station
# -----

p95_thresholds = (
    precip
    .groupby("ClimateID")["DailyPrecip_mm"]
    .quantile(0.95)
    .reset_index()
    .rename(columns={"DailyPrecip_mm": "P95_Threshold"})
)
```

```
In [5]: # -----
# 3. Merge thresholds back to daily data
# -----

precip_p95 = precip.merge(
    p95_thresholds,
    on="ClimateID",
    how="left"
)
```

```
In [6]: # -----
# 4. Filter extreme precipitation events
# -----
```

```

extreme_precip = precip_p95[
    precip_p95["DailyPrecip_mm"] >= precip_p95["P95_Threshold"]
].copy()

```

```

In [7]: # 5. Build Extreme Events records
# -----

fact_extreme_precip = extreme_precip[
    ["ClimateID", "Date/Time", "DailyPrecip_mm", "P95_Threshold"]
].copy()

fact_extreme_precip["EventType"] = "Heavy Precipitation"
fact_extreme_precip["ExtremeFlag"] = 1

# Standardize value name
fact_extreme_precip = fact_extreme_precip.rename(
    columns={"DailyPrecip_mm": "EventValue"}
)

# Reorder columns
fact_extreme_precip = fact_extreme_precip[
    [
        "ClimateID",
        "Date/Time",
        "EventType",
        "EventValue",
        "P95_Threshold",
        "ExtremeFlag"
    ]
]

```

```

In [8]: # -----
# 6. Sanity checks
# -----

print("Number of extreme precipitation events:", len(fact_extreme_precip))
print("\nEvents by season (sample):")
print(fact_extreme_precip.groupby("ClimateID").size().head())

assert (
    fact_extreme_precip["EventValue"]
    >= fact_extreme_precip["P95_Threshold"]
).all(), "Error: event below P95!"

```

Number of extreme precipitation events: 35333

Events by season (sample):

```

ClimateID
8100300    476
8100467    546
8100468    580
8100506    134
8100885    398
dtype: int64

```

```
In [9]: ### # -----
# 7. Export for APPEND
# -----

fact_extreme_precip.to_csv(
    "../Capstone/clean data/Fact_Extreme_Events_Precip.csv",
    index=False
)
```

```
In [10]: extreme_wind = pd.read_csv("../Capstone/clean data/Fact_Extreme_Events.csv")
extreme_precip = pd.read_csv("../Capstone/clean data/Fact_Extreme_Events_Precip.csv")

fact_extreme_all = pd.concat(
    [extreme_wind, extreme_precip],
    ignore_index=True
)

fact_extreme_all.to_csv(
    "../Capstone/clean data/Fact_Extreme_Events_All.csv",
    index=False
)
```

```
In [11]: fact_extreme_precip.groupby("ClimateID").size().describe()
```

```
Out[11]: count    125.000000
mean      282.664000
std       157.792276
min        1.000000
25%       150.000000
50%       266.000000
75%       405.000000
max       581.000000
dtype: float64
```

```
In [12]: fact_extreme_precip["Year"] = fact_extreme_precip["Date/Time"].dt.year
fact_extreme_precip.groupby("Year").size().tail(10)
```

```
Out[12]: Year
2017    1406
2018    1868
2019    1678
2020    1244
2021    1686
2022    1986
2023    2224
2024    1653
2025    1458
2026     327
dtype: int64
```

```
In [ ]:
```